Faculty of Engineering and Technology
B.Tech (CSE) Programme
Semester V
Subject: Artificial Intelligence [2601504]
MODULE 2

Knowledge Representation: Representations and Mappings, Approaches to Knowledge Representation.
Using Predicate Logic: Representing Simple Facts in Logic, Representing Instance and ISA Relationships, Computable Functions and Predicates, Resolution, Natural Deduction.
Using Rules: Procedural Versus Declarative Knowledge, Logic Programming, Forward Versus Backward Reasoning, Matching, and Control Knowledge.

Knowledge Representation (KR) is the method of representing knowledge in a form that a computer can understand and use to solve problems, answer questions, and make decisions.

- **Human brain** → Stores knowledge naturally through learning and experience.
- **Computer** → Cannot understand knowledge on its own.

So, **knowledge representation acts like the computer's way of organizing knowledge**, allowing it to reason and solve problems.

Knowledge +Reasoning = Intelligence

Knowledge - Facts, information, or skills acquired through education and experience.
Reasoning - The process of thinking logically using knowledge to reach a conclusion.
Intelligence - The ability to use knowledge and reasoning to make correct decisions and solve problems.

Example: Google Maps

- **Knowledge:** One road has heavy traffic.
- **Reasoning:** Heavy traffic will increase travel time.
- **Intelligence:** Suggest an alternate route with less traffic.

For AI,
Knowledge + Reasoning + Algorithms
↓
Intelligent Agent

Human Intelligence	Machine Intelligence
Learns through education	Learns from the data and knowledge provided
Gains experience	Has no real-life experience
Thinks independently	Cannot think on its own
Uses common sense	Uses programmed rules and algorithms
Makes decisions using knowledge and reasoning	Makes decisions only from stored knowledge and programmed logic

Mapping between Facts and Representations

A fact is converted into an internal representation (mapping), the reasoning program processes it, and the result is converted back into English so that humans can understand it.

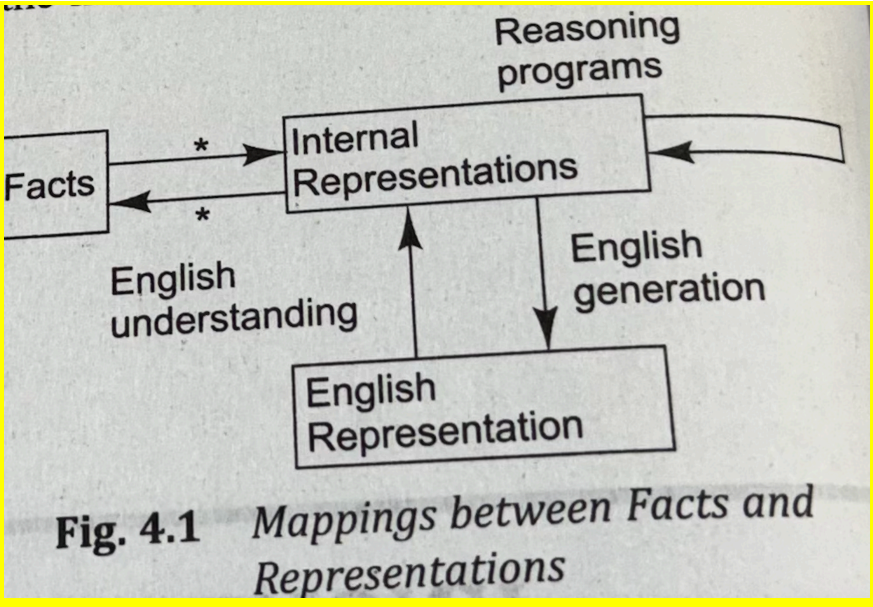


Fig. 4.1 *Mappings between Facts and Representations*

Step 1: What are Facts?

Facts are **real-world information**.

Examples:

- Ravi is a student.

- Fire is hot.
- Birds can fly.

These facts exist in the real world.

Step 2: English Understanding

Humans communicate using English (or any natural language).

Example:

> "Ravi is a student."

The computer **cannot directly understand English**.

So an **English understanding** module converts the English sentence into a computer-understandable form.

Example

English Sentence

Ravi is a student.

↓

Internal Representation

Student(Ravi)

This conversion is called **mapping from Facts to Internal Representation**.

Step 3: Internal Representation

This is the **knowledge stored inside the computer**.

The computer does **not** store:

> Ravi is a student.

Instead, it stores

Student(Ravi)

or

Attendance(Ravi,82)
Eligible(Ravi)

This is called **Internal Representation** because it is the machine's internal way of storing knowledge.

Step 4: Reasoning Programs

Now the reasoning program starts working.

Suppose the knowledge base contains

Student(Ravi)

Attendance(Ravi,82)

Rule

IF Attendance ≥ 75
THEN Eligible
The reasoning program processes the knowledge.

It concludes

Eligible(Ravi)

This is **reasoning**.

Notice that the computer **did not learn anything new from outside**. It used the stored knowledge and rules to derive a new fact.

Step 5: English Generation

Now the computer has the result

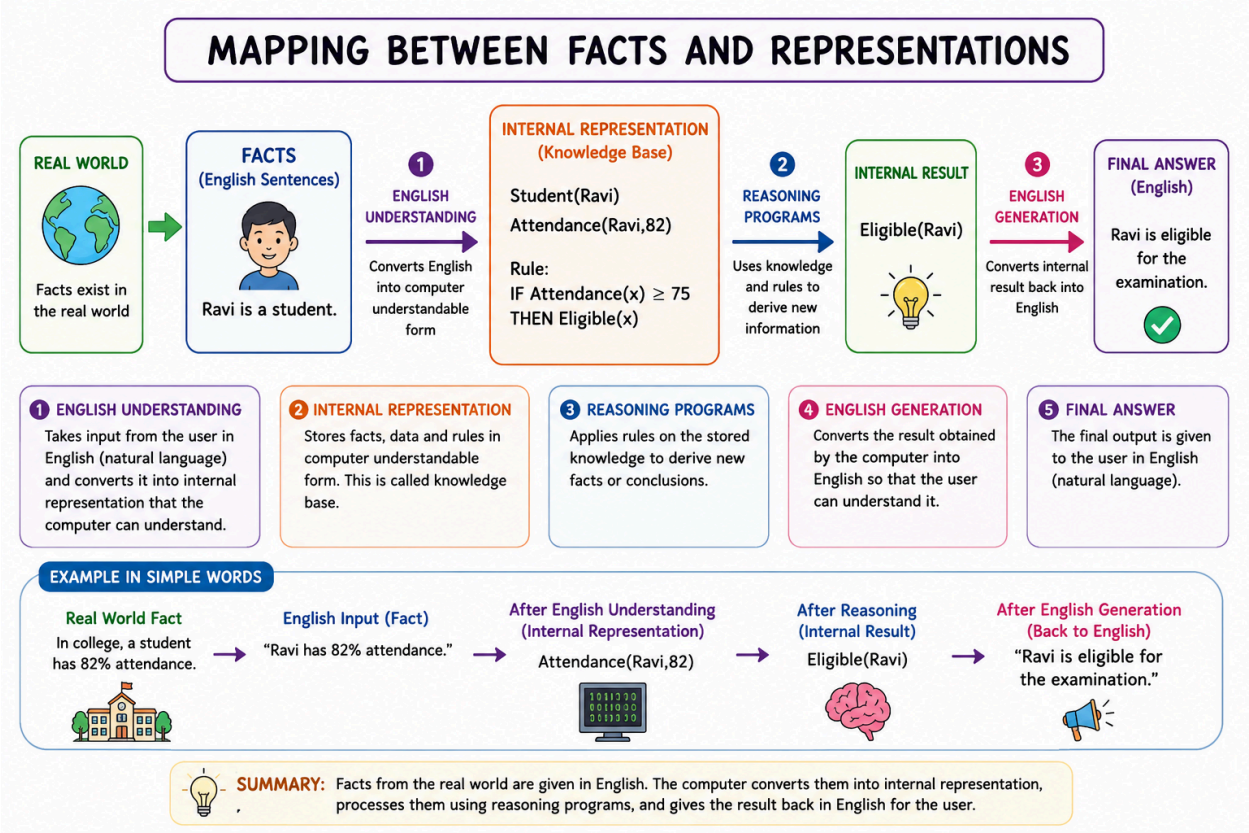
Eligible(Ravi)

Humans cannot easily understand this symbolic representation.

So the **English Generation** module converts it into a natural language sentence.

Result

Ravi is eligible for the examination.



Example (Attendance System)

English Input (Fact)
"Ravi has 82% attendance."
↓
Internal Representation (Mapping)
Attendance(Ravi, 82)
↓
Reasoning Program
Rule: IF Attendance ≥ 75%
THEN Eligible for Exam
↓
Result
Eligible(Ravi)
↓
English Output

"Ravi is eligible for the examination."

The user always gives the input in **English**. The computer cannot understand English directly, so it converts it into its own language called **internal representation** (this process is called **mapping**). Then the **reasoning program** processes the information, finds the result, and finally converts the result back into **English** so that the user can easily understand it.

Example (Google Maps)

Suppose you ask Google Maps:

> **"Show me the fastest route to college."**

- **Fact (Input):** Show me the fastest route to college.
- **Mapping (Internal Representation):** Google Maps converts your request into computer data (location, destination, roads).
- **Reasoning Program:** It checks traffic, distance, and road conditions.
- **English Generation (Output):** It displays **"Take SG Highway. Arrival time: 20 minutes."**

Representation of Facts

1. **Forward Representation Mapping:** Converts real-world facts into a machine-understandable form.
2. **Backward Representation Mapping:** Converts the machine's internal result into human-understandable language.

EXAMPLE : Weather App

Fact:

> The temperature is **38°C**.

The computer converts it into: Temperature(38)

This is **Forward Representation Mapping**.

After processing, the computer concludes:

HotWeather

It then displays: **Today's weather is hot.**

This is **Backward Representation Mapping**.

Properties of a Good Knowledge Representation System

Property	Meaning
Representational Adequacy	Store the knowledge
Inferential Adequacy	Think using the knowledge
Inferential Efficiency	Think Fast
Acquisitional Efficiency	Update the knowledge

EXAMPLE: Google Maps

Imagine you ask Google Maps:

> "Show me the fastest route from College to Home."

For Google Maps to answer correctly, it must have these four properties.

Property	Meaning	Google Maps Example
1. Representational Adequacy	Can it store all the required knowledge?	It stores roads, traffic signals, flyovers, highways, and locations.
2. Inferential Adequacy	Can it think using the stored knowledge?	It finds the shortest and fastest route using the stored map and traffic data.

3. Inferential Efficiency	Can it give the answer quickly?	It suggests the best route in a few seconds instead of taking several minutes.
4. Acquisitional Efficiency	Can it easily update new knowledge?	If a new road or bridge is built, Google Maps can update it in its database.

1. Representational Adequacy (Store Knowledge)

Imagine Google Maps knows only Ahmedabad roads but not Gandhinagar roads.

Can it guide you to Gandhinagar?

No.

Why?

Because it doesn't have that knowledge.

Representational Adequacy means the system should store all the required knowledge.

2. Inferential Adequacy (Think)

Google Maps already knows:

- Road A = 20 km
- Road B = 18 km
- Road B has heavy traffic

It thinks:

Road A will take less time.

This ability to use stored knowledge to reach a conclusion is Inferential Adequacy.

3. Inferential Efficiency (Think Fast)

Suppose Google Maps takes 30 minutes to find the route.

Would you use it?

No.

You expect the answer in 2–3 seconds.

Inferential Efficiency means giving the correct answer quickly.

4. Acquisitional Efficiency (Easy to Update)

A new flyover opens in your city.

Google Maps should allow engineers to add the new road easily.

It should not require building the whole system again.

Acquisitional Efficiency means adding or updating knowledge easily.

Knowledge Representation Schemes

Knowledge Representation is the method of storing knowledge in a computer so that it can reason, solve problems, and make decisions like a human.

The four major KR schemes are:

1. **Relational Knowledge:** Represents knowledge as relationships among objects or facts, similar to tables in a database.
2. **Inheritable Knowledge:** Organizes knowledge in a hierarchy where child objects inherit properties from parent objects.
3. **Inferential Knowledge:** Uses logical rules to derive new knowledge from existing facts.
4. **Procedural Knowledge:** Represents knowledge as procedures or sequences of steps for performing tasks.

Relational Knowledge

Relational knowledge represents knowledge in the form of **relationships among different objects or facts**.

The computer stores facts without explaining how they are connected logically.

It stores **facts that describe how two or more objects are related.**

General form:

Object 1 → Relationship → Object 2

For example:

- Aditi → teaches → AI
- Ahmedabad → is_in → Gujarat
- Cat → eats → Fish

Each of these is a **relationship (relation)**.

Example

Bird	Can Fly
Sparrow	Yes
Penguin	No

Relation:

Bird → Ability → Fly

Advantages

- Simple to understand.
- Easy to store.
- Similar to database tables.

Disadvantages

- Difficult to perform reasoning.
- Cannot represent complex knowledge.

2. Inheritable Knowledge: **Objects inherit properties from their parent class.**

Inheritable knowledge organizes information into a hierarchy. Lower-level objects automatically inherit the properties of higher-level objects.

This avoids storing the same information repeatedly.

Example

Vehicle
|
Car
|
Electric Car

Vehicle
- Has wheels

Car
- Has engine

Electric Car
- Has battery

Electric Car automatically inherits:
- Wheels
- Engine

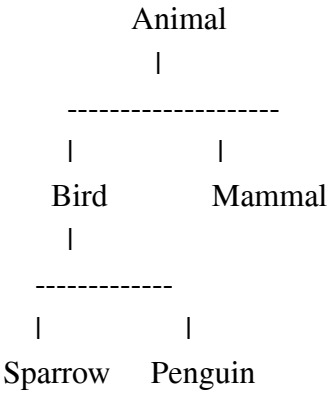
Advantages

- Reduces duplication.
- Easy to organize knowledge.
- Supports inheritance.

Disadvantages

- Handling exceptions (e.g., Penguin is a bird but cannot fly) can be difficult.

  Example



Properties:

Animal

- Living

Bird

- Has wings
- Lays eggs

Sparrow

- Can fly

Penguin

- Cannot fly

Sparrow automatically inherits:

- Living
- Has wings
- Lays eggs

3. Inferential Knowledge: Inferential knowledge represents knowledge using logical rules. New knowledge is derived (inferred) from existing facts.

Example

Facts

- Fever
- Cough

Rule

IF Fever AND Cough
THEN Flu

Inference : Patient has Flu.

Example

Stored Facts

- It is raining.
- If it rains, carry an umbrella.

Inference

The computer reasons:
"It is raining."
↓
"Rule says carry an umbrella."
↓
New Knowledge: Asha should carry an umbrella.

Note: **Inference itself is not the stored knowledge.** It is the **process of generating new knowledge** using facts and rules. The word **Inference** means **finding or deriving new knowledge** from the knowledge that is already stored.

Advantages

- Produces new knowledge.
- Supports intelligent decision-making.
- Reduces data storage.

Disadvantages

- Requires correct logical rules.
- Reasoning can be slow for large knowledge bases.

Procedural Knowledge

Procedural knowledge describes **how to perform a task**. It stores procedures, algorithms, or sequences of actions instead of facts.

Procedural knowledge stores the steps to solve a problem.

Example

Robot Cleaning Procedure

1. Start robot.
2. Scan room.
3. Detect obstacles.
4. Move forward.
5. Clean floor.
6. Return to charging station.

Example

ATM Cash Withdrawal

Step 1 → Insert card
↓
Step 2 → Enter PIN

↓
Step 3 → Select Withdrawal
↓
Step 4 → Enter Amount
↓
Step 5 → Collect Cash

Advantages

- Excellent for task execution.
- Easy to automate.
- Suitable for robots and expert systems.

Disadvantages

- Difficult to modify.
- Less flexible than declarative knowledge.

TASK (2 MARKS)

Why is procedural knowledge less flexible?

Because it follows a **fixed sequence of steps**.

Example

ATM Withdrawal Procedure

1. Insert card.
2. Enter PIN.
3. Select Withdrawal.
4. Enter amount.
5. Collect cash.

The computer must follow these steps in order.

It cannot suddenly jump to Step 4.

So, procedural knowledge is **less flexible**.

Why is declarative knowledge easier to modify?

Declarative knowledge stores **facts**, not steps.
Example:

- Tea contains milk.
- Black tea does not contain milk.

If something changes, you only update the fact.
Example:
Old fact:

College timing = 9:00 AM

New fact:

College timing = 8:30 AM

Just replace one fact.
No procedure needs to be rewritten.

Why is procedural knowledge difficult to modify?

Because knowledge is stored as a sequence of steps. If one step changes, the procedure must be updated, and changes may affect the remaining steps.

Why is procedural knowledge less flexible than declarative knowledge?

Because procedural knowledge follows a fixed sequence to perform a task, whereas declarative knowledge stores facts that can be used in many different situations without changing the stored information.

Declarative Knowledge	**Procedural Knowledge**
Tells **what is true**.	Tells **how to perform a task**.
Stores **facts and information**.	Stores **steps, procedures, or algorithms**.
Answers **"What?"**	Answers **"How?"**

Easy to update or modify by changing facts.	Difficult to modify because the sequence of steps may need to be changed.
More flexible, as the same facts can be used for different purposes.	Less flexible, as it follows a fixed sequence of steps.
Example: **Ravi studies AI.**	Example: **Steps to withdraw money from an ATM.**
Easy to modify and easy to understand.	High efficiency and difficult to modify.

Declarative Knowledge (WHAT) : Declarative knowledge tells WHAT is true by storing facts.

Declarative knowledge stores **facts**. It tells the computer **what is true**, but not how to perform a task.

Example
A school database stores:

- Asha studies AI.
- AI is a Computer Science subject.
- Asha's attendance is 82%.

These are only **facts**.

The computer knows **what** is true.

Example

Think of a **dictionary**.
A dictionary tells you the meaning of a word.
It does **not** tell you how to use the word in a sentence.
It only stores information.

Procedural Knowledge (HOW)

Procedural knowledge tells HOW to perform a task by storing steps or procedures.

Procedural knowledge stores **steps or procedures**.

It tells the computer **how to perform a task**.

Example

Steps to withdraw money from an ATM

1. Insert ATM card.
2. Enter PIN.
3. Select Withdrawal.
4. Enter amount.
5. Collect cash.

These are instructions.

Example

- Think of a **recipe**.
- A recipe tells you **how to cook**.
- It gives the steps one by one.

Forward Reasoning (Forward Chaining)	**Backward Reasoning (Backward Chaining)**
Starts with **known facts**.	Starts with a **goal or conclusion**.
Applies rules to derive a conclusion.	Works backward to verify whether the goal is true.
Direction: **Facts → Rules → Conclusion**	Direction: **Goal → Rules → Facts**
Also called **Data-Driven Reasoning** because it starts with available data (facts).	Also called **Goal-Driven Reasoning** because it starts with a specific goal.

Best used when all facts are available and we want to discover the result.	Best used when we want to prove or verify a specific goal.
Example: Fact: It is raining. Rule: IF it is raining THEN carry an umbrella. **Conclusion:** Carry an umbrella.	**Example:** Goal: Is Asha eligible for the exam? Rule: IF attendance ≥ 75% THEN eligible. The system checks Asha's attendance (82%) and concludes that Asha is eligible.
Example: Medical diagnosis – Fever + Cough → Flu.	**Example:** Bank loan approval – Check income, credit score, and employment to decide whether the loan can be approved.

Control Knowledge

When/Which Next? (Order of steps)

Control Knowledge is the knowledge that guides an AI system in selecting and ordering the actions or rules to solve a problem efficiently. It determines what to do next during the reasoning or search process.

Example : AI Robot

A robot has to clean a room.

It knows these actions:

- Pick up toys
- Sweep the floor
- Mop the floor

Control Knowledge tells the robot:

1. Pick up the toys first.

2. Sweep the floor next.
3. Mop the floor last.

If the robot mops first and then picks up toys, it will have to clean again!

Example: Traffic Signal

A traffic signal follows these rules:

- Red → Stop
- Yellow → Get Ready
- Green → Go

The signal tells drivers which action to perform at the right time.

This is similar to Control Knowledge, because it controls when each action should happen.

Logic Programming

Logic Programming is a simple way of solving problems using facts and rules.

- Fact: Something that is true.
- Rule: A condition that tells us something is true.
- Query: A question asked to the computer.
- The computer uses logic to find the answer.

Simple Formula:

Facts + Rules + Query → Answer

Example:

Facts:

- Ram is a student.
- Ram studies AI.

Rule:

- If a person is a student, then the person studies.

Query:
Is Ram a student?

Answer: Yes. ✅

Note:
Logic Programming = Facts + Rules + Questions → Answer

Prolog is a popular programming language used for logic programming in AI.

Propositional Logic

- **No variables**
- Complete statement
- Always **True or False**

Examples:

- 20<10
- Ahmedabad is in Gujarat.
- Today is Monday.

Predicate Logic

- **Contains variables**
- Describes properties or relationships
- Truth depends on the variable

Examples:

- X<10
- Student(X)
- Age(X)>18

Step 1: Consider one simple sentence

Sentence:

Ravi is a student.

In Propositional Logic

We simply write

P = Ravi is a student

That's it.

The computer only knows:

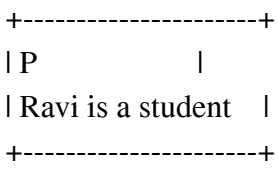
P = True
or
P = False

It **does not know**:

- Who is Ravi?
- What is a student?
- Is Ravi a person?
- Can Ravi have other properties?

It only knows **P**.

Think of it as a **closed box**.



The computer cannot look inside the box.

In First-Order Predicate Logic

Instead of storing the whole sentence as one symbol, we divide it.

Student(Ravi)

Now the computer knows

Student → Property
Ravi → Person

It understands

Ravi belongs to the Student category.

Instead of one box, everything is visible.

Student
 |
 |
Ravi

This is why FOPL is smarter.

Step 2: Add another sentence

Now suppose we say

Ravi studies.

Propositional Logic

P = Ravi is a student
Q = Ravi studies

The computer sees

P
Q

Question:

Does it know that Ravi in P and Ravi in Q are the same person?

❌ No.

For the computer,

P
Q

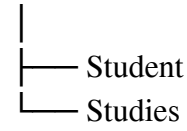
are just two unrelated statements.

FOPL

Student(Ravi)

Studies(Ravi)

Now the computer immediately knows

Ravi


It understands both facts are about **the same Ravi**.

Step 3: Now add one more student

Suppose

- Ravi studies.
- Priya studies.
- Amit studies.

Propositional Logic

Need three statements.

P = Ravi studies

Q = Priya studies

R = Amit studies

The computer stores

P
Q
R

Nothing connects them.

FOPL

Studies(Ravi)

Studies(Priya)

Studies(Amit)

Now it understands

Studies

├── Ravi
├── Priya
└── Amit

It knows **Studies** is the same property for all three people.

Step 4: Now suppose a teacher says

> Every student studies.

This is where the biggest difference comes.

Propositional Logic

It cannot write

> Every student studies.

So we must write

Ravi studies
Priya studies
Amit studies
Neha studies
Karan studies
...

If there are 500 students,

you need 500 statements.

FOPL

Just one statement.

$\forall x \, (Student(x) \rightarrow Studies(x))$

Meaning

If anyone is a student,
then that person studies.

One sentence covers everyone.

Step 5: Reasoning

Suppose we know

Every student studies.

Ravi is a student.

Propositional Logic

It has

P = Every student studies
Q = Ravi is a student

Can it conclude

Ravi studies

❌ No.

Because P and Q are just two separate labels.

FOPL

Knowledge

Student(Ravi)

$\forall x(Student(x) \rightarrow Studies(x))$

The computer thinks

Rule:

IF Student

↓

THEN Studies

Ravi is Student

↓

Therefore

Ravi Studies

It automatically concludes

Studies(Ravi)

This is called **logical inference**.

An Easy Analogy

Imagine three student ID cards.

Propositional Logic

It only stores the envelopes.

Envelope P

Envelope Q

Envelope R

It cannot see what is inside.

First-Order Predicate Logic

It opens every envelope.

Name : Ravi

Profession : Student

Age : 20

Now it knows

- Who?
- What?
- Relationship?

Everything.

Difference (5 Marks)

Propositional Logic	First-Order Predicate Logic
Stores the **whole sentence** as one symbol (P, Q, R).	Breaks the sentence into **object + property + relationship**.
Knows only **True/False**.	Knows **who**, **what**, and **how they are related**.
Cannot write general rules like "Every student studies."	Can write one rule using ∀ (for all).
Cannot derive many new facts from general rules.	Can derive new facts by applying rules to specific objects.

Propositional Logic = Whole Sentence

Example:

- **P = Ravi studies.**

It only knows **P**.

First-Order Predicate Logic = Sentence is Broken into Meaningful Parts

Example:

- **Student(Ravi)**
- **Studies(Ravi)**

- **Likes(Ravi, Cricket)**

It knows:

- **Who?** → Ravi
- **What property?** → Student
- **What action?** → Studies
- **Relationship?** → Likes Cricket

Note:
Propositional Logic only knows whether a statement is true or false. First-Order Predicate Logic knows who, what, and how things are related, making it much more powerful for Artificial Intelligence.

Difference Between → and ↔ (2 Marks)

Implication (→)	**If and Only If (↔)**
One-way relationship	Two-way relationship
If P, then Q	P if and only if Q
P → Q	P ↔ Q
Reverse may not be true	Reverse is always true

Task: Convert the following statements into First-Order Logic

1. Every student studies.
2. Every teacher teaches a subject.
3. Some students play cricket.
4. Every bird can fly.
5. Some birds cannot fly.
6. Every doctor treats patients.
7. Some doctors work in a hospital.
8. Every student who studies regularly passes the examination.
9. No student is absent from every class.
10. Every person has a name.
11. Some students like mathematics.
12. Every computer has a processor.
13. Every employee works in a company.

14. Some employees work from home.
15. Every customer who buys a product makes a payment.
16. No teacher is a student.
17. Some students are also programmers.
18. Every parent has a child.
19. Every engineer has technical knowledge.
20. There exists a student who knows Python.

Components of First-Order Predicate Logic (FOPL)

First-Order Predicate Logic (FOPL) represents objects, their properties, and relationships. It has the following basic components:

1. Constants

A constant represents a specific object, person, place, or thing.

Examples

- John
- Ahmedabad
- India
- Laptop1

Predicate Example:

Student(John)

Here, John is a constant.

2. Variables

A variable represents any object.

Variables are usually written as:

- X
- Y
- Z

Example

Student(X)

Meaning: X can be any student.

3. Predicates

A predicate describes a property of an object or a relationship between objects.

Property Example

Student(John)

Meaning: John is a student.

Relationship Example

WorksIn(Rahul,HR)

Meaning: Rahul works in the HR department.

4. Functions

A function returns a value related to an object.

Example

Manager(Employee101)

Meaning: The function Manager() returns the manager of Employee101.

Another example:

Father(Rahul)

Meaning: The function returns Rahul's father.

5. Quantifiers

Quantifiers specify how many objects satisfy a condition.

(a) Universal Quantifier ($\forall$)

Means Every or All.

Example:

$\forall X(Student(X) \rightarrow Studies(X))$

Meaning:

Every student studies.

(b) Existential Quantifier ($\exists$)

Means There exists or At least one.

Example:

$\exists X Student(X)$

Meaning:

There is at least one student.

6. Logical Connectives

Logical connectives combine two or more statements.

Symbol	Meaning	Example
$\wedge$	AND	$Student(X) \wedge FeesPaid(X)$
$\vee$	OR	$Student(X) \vee Teacher(X)$
$\neg$	NOT	$\neg Student(X)$
$\rightarrow$	IF...THEN	$Student(X) \rightarrow Studies(X)$
$\leftrightarrow$	IF AND ONLY IF	$Pass(X) \leftrightarrow Eligible(X)$

Representing Simple Facts in Logic

Definition: Representing simple facts means converting statements about objects into logical form.

A fact tells us something that is true.

Example 1

English:

Ravi is a student.

Logic:

Student(Ravi)

Meaning: Ravi has the property Student.

Example 2

English:

Ravi likes AI.

Likes(Ravi,AI)

Meaning: There is a relationship Likes between Ravi and AI.

Example 3

English:

Amit teaches Python.

Teaches(Amit,Python)

Representing Instance Relationships

Definition: An instance relationship tells us that a particular object belongs to a class or category.

Example

Ravi is a student.

Here:

- Ravi = Object
- Student = Class

So:

Student(Ravi)

We can say:

> Ravi is an instance of Student.

Another example

> Tata Nexon is a car.

Car(TataNexon)

Meaning:

> Tata Nexon is an instance of the class Car.

Representing Instance and ISA Relationships

AI often stores objects in categories.

Two important relationships are:

- Instance Relationship
- ISA Relationship

(A) Instance Relationship

An **instance** means a specific object belongs to a class.

General Form

Instance(Object, Class)

Example

Ravi is a student.

Instance(Ravi, Student)

Meaning:

Ravi is one member of the Student class.

More examples

Instance(Dog1, Dog)

Instance(Tweety, Bird)

Instance(Car1, Car)

(B) ISA Relationship

ISA means

"Is A"

It shows one class is a subtype of another class.

General Form

ISA(Subclass, Superclass)

Example

A dog is an animal.

ISA(Dog, Animal)

Meaning

Every Dog is also an Animal.

Another Example

ISA(Student, Person)

Meaning

Every Student is a Person.

Combined Example

Instance(Ravi, Student)

ISA(Student, Person)

Inference:

Person(Ravi)

Meaning:

Since Ravi is a Student and every Student is a Person,

therefore Ravi is also a Person.

Real-Life Example

Instance(BMW, Car)

ISA(Car, Vehicle)

Inference

Vehicle(BMW)

Computable Functions and Predicates

Predicate Logic can also represent **functions** and **predicates**.

Computable Function

A function returns exactly **one value**.

Function(Object)

Example

Father of Ravi

Father(Ravi) = Mahesh

Predicate

A predicate returns **True or False**.

Example

Even(8)

Result

True

Resolution

Resolution is an **inference rule** used to derive new facts or prove whether a statement is true.

It works by combining existing facts and rules to reach a logical conclusion.

Example

Facts

Student(Ravi)

Student(x) → EligibleForExam(x)

Question

Is Ravi eligible for the exam?

Resolution Process

Step 1

Student(Ravi)

Step 2

Student(x) → EligibleForExam(x)

Replace x with Ravi.

Student(Ravi) → EligibleForExam(Ravi)

Since Student(Ravi) is true,

we conclude

EligibleForExam(Ravi)

Example 2

Rule

Bird(x) → CanFly(x)

Fact

Bird(Tweety)

Conclusion

CanFly(Tweety)

Resolution derives the new fact **CanFly(Tweety)**.

5. Natural Deduction

Natural Deduction is a reasoning method that derives conclusions using logical rules in a step-by-step manner.

It is similar to how humans naturally reason.

Example

Fact 1

Student(Ravi)

Fact 2

Student(x) → GetsIDCard(x)

Step 1

Ravi is a Student.

Step 2

Every Student gets an ID card.

Step 3

Therefore,

GetsIDCard(Ravi)

Example

Rule

Employee(x) → HasOfficeID(x)

Fact

Employee(Aditi)

Conclusion

HasOfficeID(Aditi)

Matching in Logic Programming

Matching is the process of comparing a fact, rule, or query to see whether they fit together.

It is an important part of Logic Programming, especially in languages such as Prolog.

Example

Suppose we have a fact:

Student(Rahul)

And a query:

Student(X)

The system compares them:

Student(X)
Student(Rahul)

They match if:

X = Rahul

So the answer is:

X = Rahul

Example with Two Arguments

Fact

Likes(Rahul, Cricket)

Query

Likes(Rahul, X)

The system matches:

X = Cricket

Therefore:

Answer: X = Cricket

Matching with a Rule

Suppose we have:

Fact

Student(Rahul)

Rule

Student(X) → Eligible(X)

The system tries to match:

Student(X)

with

Student(Rahul)

They match when:

X = Rahul

The rule then gives:

Eligible(Rahul)

Therefore:

Student(Rahul)
↓ Matching
X = Rahul
↓ Apply rule
Eligible(Rahul)

Real-Life Example: Online Banking

Facts

HasAccount(Amit)
HasBalance(Amit, 50000)

Rule

$\text{HasAccount}(X) \land \text{HasBalance}(X, B) \rightarrow \text{CanWithdraw}(X)$

Query

CanWithdraw(Amit)

The system matches:

HasAccount(X)

with

HasAccount(Amit)

So:

X = Amit

It also matches:

HasBalance(X, B)

with:

HasBalance(Amit, 50000)

So:

X = Amit
B = 50000

Therefore:

CanWithdraw(Amit)

Matching vs Exact Matching

Matching does **not always mean that the expressions are exactly identical**.

Variables can take suitable values.

Example

Likes(X, Cricket)

and

Likes(Rahul, Cricket)

Match with:

X = Rahul

But:

Likes(Rahul, Cricket)

and

Likes(Amit, Football)

do not match, because the arguments are different.

What is Prolog?

Prolog stands for:

PROgramming in LOGic

Unlike Python, where we usually give step-by-step instructions, in Prolog we give the computer:

1. **Facts** – what is true
2. **Rules** – what can be concluded
3. **Queries** – what we want to know

Think of it as:

Knowledge → Reasoning → Answer

Step 1: Write Your First Fact

Suppose we want to tell Prolog:

Rahul is a student.

Write:

student(rahul).

This is called a **fact**.

Important

student → Predicate (describes a property/relationship)
rahul → Constant / Atom (the name of an individual)
(rahul) → Argument
. → End of the Prolog statement
student(rahul) → A fact

The . at the end is important in Prolog.

Step 2: Ask Prolog a Question

Now ask:

student(rahul).

Prolog answers:

true.

Because we already told Prolog that Rahul is a student.

Step 3: Ask About Someone Else

Fact:

student(rahul).

Query:

student(amit).

Answer:

false.

Because we never said that Amit is a student.

Step 4: Use a Variable

Now we want to ask:

> Who is a student?

Write:

student(X).

If our facts are:

student(rahul).
student(priya).
student(amit).

Prolog gives:

X = rahul
X = priya
X = amit

This is MATCHING

Prolog matches:

student(X)

with:

student(rahul)

Therefore:

X = rahul

Step 5: Add a Relationship

Suppose:

> Rahul studies AI.

Write:

studies(rahul, ai).

Here we have **two arguments**.

studies(Person, Subject)

Ask:

studies(rahul, ai).

Answer:

true.

Ask: What does Rahul study?

Query:

studies(rahul, X).

Answer:

X = ai

Again, this is **matching**.

Step 6: Write a Rule

Now we tell Prolog:

Every student studies AI.

Rule:

studies_ai(X) :- student(X).

Read it as:

X studies AI if X is a student.

Here:

:- means "if"

So:

studies_ai(X) :- student(X).

means:

IF student(X), THEN studies_ai(X).

Step 7: Use the Rule

Facts:

student(rahul).
student(priya).

Rule:

studies_ai(X) :- student(X).

Query:

studies_ai(rahul).

Prolog checks:

Is Rahul a student?
↓
Yes
↓
Therefore Rahul studies AI
↓
true

Step 8: One Complete Small Program

This is the first program I recommend teaching your students:

student(rahul).
student(priya).
student(amit).

studies_ai(X) :- student(X).

Now try these queries:

Query 1

student(rahul).

Answer:

true

Query 2

student(rohit).

Answer:

false

Query 3

student(X).

Answer:

X = rahul
X = priya
X = amit

Query 4

studies_ai(rahul).

Answer:

true

Query 5

studies_ai(X).

Answer:

X = rahul

X = priya
X = amit

Demo : https://onecompiler.com/prolog/44y2gckpv

